

QEMU

In the previous articles, we learnt how to use QEMU for a generic Linux OS installation, for networking using OpenVPN and TAP/TUN, for cross-compilation of the Linux kernel for ARM, to boot the kernel from QEMU, and how to build a small filesystem and then mount it on the vanilla kernel. Now we will step out further.

Part—3

First of all, I would like to explain the need for a boot-loader. The boot-loader is code that is used to load the kernel into RAM, and then specify which partition will be mounted as the root filesystem. The boot-loader resides in the MBR (Master Boot Record). In general-purpose computing machines, an important component is the BIOS (Basic Input Output System). The BIOS contains the low-level drivers for devices like the keyboard, mouse, display, etc. It initiates the boot-loader, which then loads the kernel. Linux users are very familiar with boot-loaders like GRUB (Grand Unified Boot-Loader) and LILO (Linux Loader).

Micro-controller programmers are very familiar with the term 'Bare-Metal Programming'. It means that there is nothing between your program and the processor—the code you write runs directly on the processor. It becomes the programmer's responsibility to check each and every possible condition that can corrupt the system.

Now, let us build a small program for the ARM Versatile Platform Baseboard, which will run on the QEMU emulator, and then print a message on the serial console. This time, I have downloaded the tool-chain for ARM EABI from <http://bit.ly/9XbRBA>. As described in the previous articles, add this tool-chain in your PATH.

By default, QEMU redirects the serial console output to the terminal, when it is initialised with the `-nographic` option:

```
$ qemu-system-arm --help | grep nographic
-nographic    disable graphical output and redirect serial I/Os to
console. When using -nographic, press 'ctrl-a h' to get some help.
```

We can make good use of this feature; let's write some data to the serial port, and it can be a good working example.

Before going further, we must make sure which processor the GNU EABI tool-chain supports, and which processor QEMU can emulate. There should be a similar processor supported by both the tool-chain and the emulator. Let's check first in QEMU. In the earlier articles, we compiled the QEMU source code, so use that source code to get the list of the supported ARM processors:

```
$ cd (your-path)/qemu/qemu-0.14.0/hw
$ grep "arm" versatilepb.c
#include "arm-misc.h"
static struct arm_boot_info versatile_binfo;
```

```
cpu_model = "arm926";
```

It's very clear that the `arm926` is supported by QEMU. Let's check its availability in the GNU ARM tool-chain:

```
$ cd (your-path)/CodeSourcery/Sourcery_G++_lite/share/doc/arm-arm-
none-eabi/info
$ cat gcc.info | grep arm | head -n 20

        'strongarm110', 'arm8', 'arm810', 'arm9', 'arm9e',
'arm920', 'arm920t', 'arm922t', 'arm946e-s', 'arm966e-s',
'arm968e-s', 'arm926j-s', 'arm940t', 'arm9tdmi', 'arm10tdmi',
'arm1020t', 'arm1026j-s', 'arm10e', 'arm1020e', 'arm1022e',
'arm1136j-s',
```

Great!! The ARM926EJ-S processor is supported by the GNU ARM tool-chain. Now, let's write some data to the serial port of this processor. As we are not using any header file that describes the address of UART0, we must find it manually, from the file (`your-path/qemu/qemu-0.14.0/hw/versatilepb.c`):

```
/* 0x101f0000 Smart card 0. */
/* 0x101f1000 UART0. */
/* 0x101f2000 UART1. */
/* 0x101f3000 UART2. */
```

Open source code is so powerful, it gives you each and every detail. UART0 is present at address `0x101f1000`. For testing purposes, we can write data directly to this address, and check output on the terminal. Our first test program is a bare-metal program running directly on the processor, without the help of a boot-loader. We have to create three important files. First of all, let us develop a small application program (`init.c`):

```
volatile unsigned char * const UART0_PTR = (unsigned char
*)0x101f1000;

void display(const char *string){
    while(*string != '\0'){
        *UART0_PTR = *string;
        string++;
    }
}
```